Trabalho Prático: Monitoramento e Observabilidade com Zabbix e Prometheus + Grafana

Nov 12, 2026 · @Igor Luiz Oliveira de Souza

Visão geral

Duplas de alunos vão construir, no próprio computador, o laboratório virtual de uma pequena empresa com duas unidades ligadas por WAN e uma loja virtual. Depois vão monitorar esse ambiente com duas ferramentas: Zabbix e Prometheus + Grafana. A nota vai de 0 a 10. Ela é calculada sobre 100 pontos, com até 10 pontos de bônus, e é dada no dia da apresentação, com falhas injetadas ao vivo.

- **Disciplina:** Projeto e Administração de Redes
- **Curso:** Bacharelado em Sistemas de Informação, IFBA, campus Vitória da Conquista
- **Modalidade:** trabalho em dupla (sempre 2 alunos), com arguição individual
- **Duração sugerida:** 4 semanas, com apresentações a partir da semana 4

Objetivos de aprendizagem. Ao final, o aluno deve ser capaz de:

1. Projetar uma topologia com segmentação, plano de endereçamento IP e roteamento estático entre as unidades.
2. Implantar o Zabbix para monitorar rede e servidores por agente, SNMP, ICMP e cenários web.
3. Implantar Prometheus, exporters, Alertmanager e Grafana para métricas de infraestrutura e de aplicação.
4. Diferenciar monitoramento de observabilidade e usar os três sinais (métricas, logs e traces) e os métodos Golden Signals, RED e USE.
5. Definir SLOs e alertas acionáveis e diagnosticar a causa raiz de uma falha a partir dos dados coletados.
6. Comparar criticamente as duas abordagens: coleta push x pull, agente x exporter, modelo de dados e casos de uso.

O desafio

A dupla foi contratada para tornar observável a infraestrutura da **Café Planalto**, uma torrefação fictícia de Vitória da Conquista que vende café especial pela internet. A empresa tem uma **matriz**, onde ficam os servidores e a loja virtual, e uma **filial** de atendimento, ligada à matriz por um enlace WAN.

No último mês, a loja ficou fora do ar duas vezes e a diretoria só soube pelos clientes. Em uma das vezes, o problema era o banco de dados. Na outra, era o enlace entre as unidades. Ninguém conseguiu dizer quando a falha começou nem onde estava.

A missão da dupla:

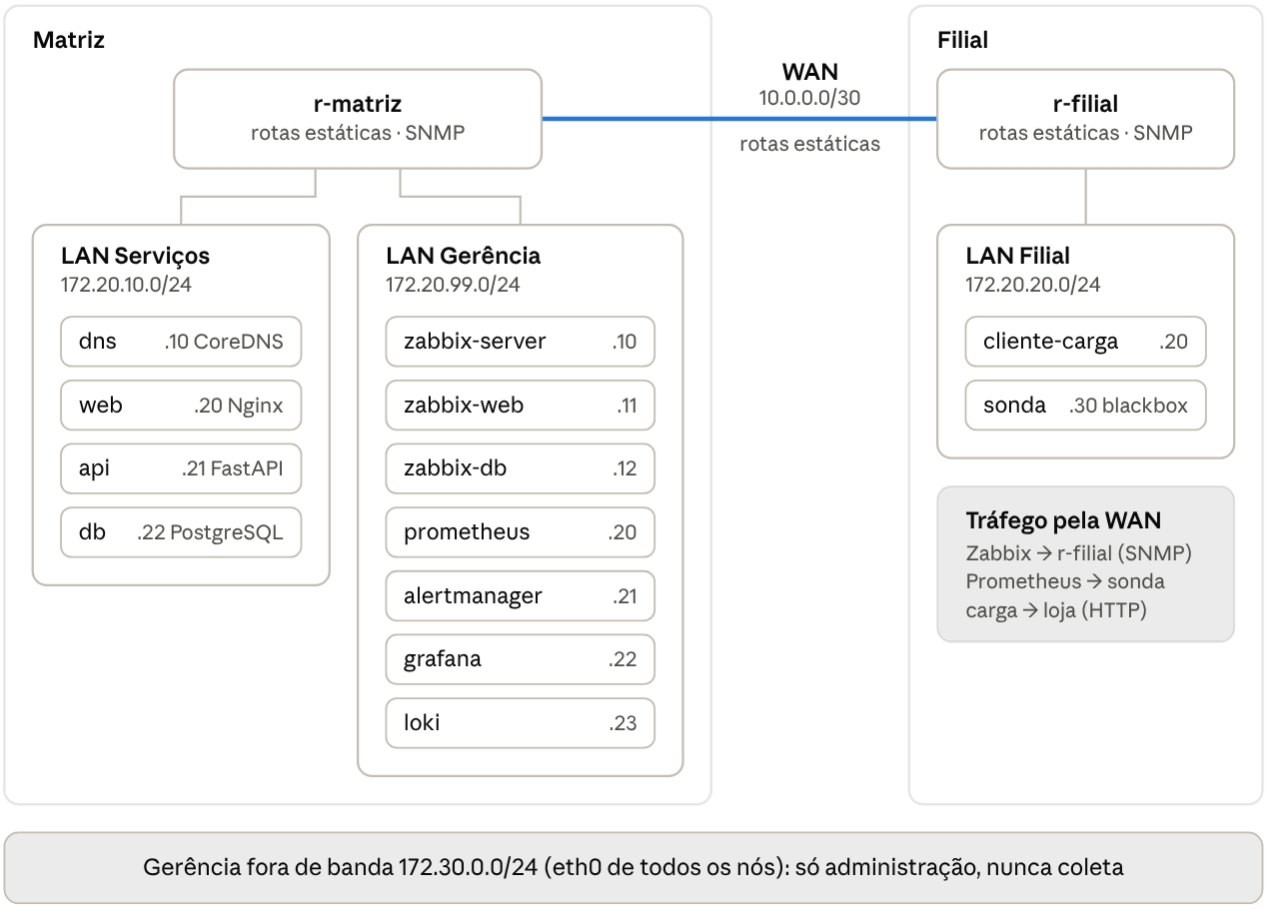
1. Reproduzir a infraestrutura da Café Planalto em um ambiente virtual, descrito como código e que sobe com um comando.
2. Implantar o **Zabbix** como ferramenta de monitoramento da infraestrutura e da rede (roteadores, enlaces, servidores, serviços).
3. Implantar **Prometheus + Grafana** (com Alertmanager e Loki) como plataforma de observabilidade da aplicação e dos contêineres.
4. Definir SLOs para a loja e alertas que avisem a dupla **antes** do cliente.
5. No dia da apresentação, o professor vai provocar falhas sorteadas. A dupla deve detectá-las, localizá-las e explicar a causa raiz usando apenas as ferramentas implantadas.
6. Entregar um relatório que recomende à diretoria como usar as duas ferramentas, com base no que a dupla observou.

As duas ferramentas devem monitorar o **mesmo** ambiente. Elas se sobrepõem de propósito: a comparação entre elas é parte da avaliação.

Infraestrutura sugerida

O laboratório inteiro roda em contêineres dentro de **uma única VM Linux**, orquestrado pelo **Containerlab**. Assim, cada roteador, switch e servidor vira um nó leve, e a topologia fica descrita em um arquivo YAML versionado no Git. Isso funciona em Windows, Linux e macOS com 8 GB de RAM livres, o que máquinas virtuais completas para cada nó não permitiriam.

Matriz e filial ligadas por rotas estáticas; a coleta atravessa a WAN



topologia do laboratório · 2 roteadores, 3 LANs, 13 hosts

O enlace WAN em destaque é o ponto crítico: a sonda e o gerador de carga ficam na filial, e o Zabbix monitora o r-filial por ele, então tudo o que a filial mede passa por ele.

Plataforma

Camada	Recomendação	Observação
Hipervisor	VirtualBox (Windows/Linux), UTM (macOS) ou WSL2 (Windows)	Uma VM só para o laboratório
Sistema da VM	Ubuntu Server 24.04 LTS	4 vCPU, 8 GB de RAM, 40 GB de disco
Contêineres	Docker Engine	Imagens oficiais de cada ferramenta
Topologia	Containerlab	Enlaces ponto a ponto e bridges definidos em YAML
Roteadores	Contêiner Linux (Alpine) com net-snmp	Encaminhamento IP, rotas estáticas e SNMP para o Zabbix

Camada	Recomendação	Observação
Switches	Bridge Linux	Um por segmento de LAN

Hardware do aluno. O ideal é um host com 16 GB de RAM. Com 8 GB, o host deve rodar Linux nativo ou a dupla deve concentrar o laboratório na máquina mais forte (ou nos computadores do laboratório do campus). O consumo estimado do ambiente completo é de 4 a 5 GB de RAM; a maior parte vem do Zabbix com seu banco e do Prometheus.

Apple Silicon (M1 a M4). Use apenas imagens com versão arm64. Alpine, Zabbix, Prometheus, Grafana e Loki têm essa versão. Imagens de roteadores como o VyOS não têm, por isso os roteadores são contêineres Linux comuns.

Alternativa permitida. Duplas podem usar GNS3 ou EVE-NG com VMs de roteador, ou Docker Compose com um contêiner roteador por rede. A topologia, o plano de endereçamento e os requisitos continuam os mesmos.

Plano de endereçamento

Segmento	Rede	Elementos e endereços
WAN matriz-filial	10.0.0.0/30	r-matriz .1, r-filial .2
LAN Serviços (matriz)	172.20.10.0/24	gateway .1, dns .10, web .20, api .21, db .22
LAN Gerência (matriz)	172.20.99.0/24	gateway .1, zabbix-server .10, zabbix-web .11, zabbix-db .12, prometheus .20, alertmanager .21, grafana .22, loki .23
LAN Filial	172.20.20.0/24	gateway .1, cliente-carga .20, sonda (blackbox exporter) .30
Gerência fora de banda (Containerlab)	172.30.0.0/24	eth0 de todos os nós, só para administração e acesso à internet

Regra importante: o tráfego de monitoramento deve passar pela rede de dados do laboratório (eth1 em diante), nunca pela rede de gerência fora de banda. Se a coleta usar o eth0, a queda do enlace WAN fica invisível e o cenário de falha perde o sentido. Cada host recebe uma rota para 172.20.0.0/16 via o roteador da sua LAN.

Elementos de rede

- **r-matriz:** roteador da matriz. Encaminha o tráfego entre a WAN e as duas LANs da matriz usando rotas estáticas. Expõe SNMP v2c.

- **r-filial:** roteador da filial, com rotas estáticas para as LANs da matriz e SNMP.
- **sw-servicos, sw-gerencia, sw-filial:** um switch (bridge Linux) por LAN.
- **dns:** CoreDNS com a zona `cafeplanalto.lab`. A loja é acessada por `loja.cafeplanalto.lab`.
- **sonda:** blackbox exporter na filial. Mede a loja e o DNS do ponto de vista de um cliente remoto.
- **cliente-carga:** gera tráfego contínuo para a loja a partir da filial.

Tabela de rotas estáticas

Não há roteamento dinâmico. Cada rota é configurada à mão no arquivo de topologia, e os roteadores precisam de `net.ipv4.ip_forward=1`.

Equipamento	Destino	Próximo salto
r-matriz	172.20.20.0/24 (LAN Filial)	10.0.0.2
r-filial	172.20.10.0/24 (LAN Serviços)	10.0.0.1
r-filial	172.20.99.0/24 (LAN Gerência)	10.0.0.1
Hosts de cada LAN	172.20.0.0/16 e 10.0.0.0/30	Gateway .1 da própria LAN

Exemplo de topologia (trecho)

```
name: cafeplanalto
mgmt:
  network: clab-mgmt
  ipv4-subnet: 172.30.0.0/24
topology:
  nodes:
    r-matriz:
      kind: linux
      image: cafeplanalto/roteador:latest   # Alpine + net-snmp, Dockerfile
da dupla
      exec:
        - sysctl -w net.ipv4.ip_forward=1
        - ip addr add 10.0.0.1/30 dev eth1
        - ip addr add 172.20.10.1/24 dev eth2
        - ip addr add 172.20.99.1/24 dev eth3
        - ip route add 172.20.20.0/24 via 10.0.0.2
    r-filial:
      kind: linux
      image: cafeplanalto/roteador:latest
```

```
      exec:
        - sysctl -w net.ipv4.ip_forward=1
        - ip addr add 10.0.0.2/30 dev eth1
        - ip addr add 172.20.20.1/24 dev eth2
        - ip route add 172.20.10.0/24 via 10.0.0.1
        - ip route add 172.20.99.0/24 via 10.0.0.1
    sw-servicos:
      kind: bridge                          # criar a bridge no host antes do
deploy
    api:
      kind: linux
      image: cafeplanalto/api:latest
      exec:
        - ip addr add 172.20.10.21/24 dev eth1
        - ip route add 172.20.0.0/16 via 172.20.10.1
        - ip route add 10.0.0.0/30 via 172.20.10.1
  links:
    - endpoints: ["r-matriz:eth1", "r-filial:eth1"]    # WAN
    - endpoints: ["r-matriz:eth2", "sw-servicos:e1"]
    - endpoints: ["api:eth1", "sw-servicos:e2"]
```

O arquivo completo, os Dockerfiles e as configurações ficam no repositório da dupla. O ambiente deve subir com um único comando (por exemplo, `make up`, que cria as bridges e roda `containerlab deploy`).

A aplicação: Loja Café Planalto

A dupla desenvolve uma loja virtual simples em três camadas, na linguagem que preferir (sugestão: Python com FastAPI ou Node.js com Express). A loja não precisa ser bonita. Ela precisa ser **observável** e **quebrável sob comando**.

Componentes

Componente	Função	Tecnologia sugerida
web	Proxy reverso e páginas estáticas	Nginx com `stub_status` habilitado
api	Catálogo e pedidos	FastAPI ou Express
db	Produtos e pedidos	PostgreSQL
cliente-carga (filial)	Simula clientes comprando o tempo todo	k6 ou Locust em loop

Endpoints obrigatórios da API

- `GET /produtos`: lista o catálogo.
- `POST /pedidos`: cria um pedido e grava no banco.
- `GET /health`: retorna 200 apenas se o banco responde.
- `GET /metrics`: métricas no formato do Prometheus.
- `POST /admin/caos`: liga e desliga falhas. Deve aceitar ao menos `latencia_ms` (atraso artificial) e `taxa_erro` (fração de respostas 500).

Os três sinais de observabilidade

Métricas (obrigatório). A API deve expor, no mínimo:

- `http_requests_total`, com os labels `rota`, `metodo` e `status`;
- `http_request_duration_seconds`, um histograma por rota;
- `pedidos_criados_total` e `pedidos_valor_reais_total`, as métricas de negócio;
- `db_pool_conexoes_em_uso`, um gauge.

Não use identificadores de usuário ou de pedido como label. Isso explode a cardinalidade do Prometheus.

Logs (obrigatório). Logs em JSON na saída padrão, com `timestamp`, `nivel`, `rota`, `status`, `duracao_ms` e `trace_id` (quando houver). Os logs devem ser coletados pelo Loki.

Traces (bônus). Instrumentação com OpenTelemetry e envio para Grafana Tempo ou Jaeger, mostrando o caminho web → api → db de uma requisição lenta.

Carga contínua

O cliente-carga roda na filial e acessa `loja.cafeplanalto.lab` pela WAN, gerando de 5 a 20 requisições por segundo com uma mistura de consultas e pedidos. Sem tráfego, os painéis ficam vazios e as falhas não aparecem.

Alternativa para duplas com pouca experiência em desenvolvimento. A dupla pode usar o podinfo como api, que já expõe métricas e endpoints de atraso e erro. Mesmo assim, precisa acrescentar o banco e ao menos uma métrica de negócio. Nesse caso, o item de instrumentação própria do barema vale metade.

Requisitos obrigatórios

Os requisitos têm códigos (I, A, Z, P, O) que se repetem no barema de avaliação. Toda configuração deve estar no repositório: arquivos de configuração, regras, dashboards em JSON e templates exportados. O que foi feito só clicando na interface e não está versionado vale metade.

I. Infraestrutura do laboratório

- **I1.** Topologia da seção anterior funcionando com roteamento estático: da filial, o `traceroute` até a loja passa por r-filial e r-matriz. As rotas são criadas pelo arquivo de topologia, não digitadas à mão depois que o ambiente sobe.
- **I2.** DNS interno resolvendo os nomes de todos os hosts e da loja.
- **I3.** O ambiente sobe do zero com um comando e está descrito no README.
- **I4.** Diagrama da topologia, plano de endereçamento e tabela de rotas no relatório.

A. Aplicação

- **A1.** Loja com web, api e db funcionando, acessível por `loja.cafeplanalto.lab` a partir da filial.
- **A2.** Métricas, logs em JSON, `/health` e `/admin/caos` como descritos na seção da aplicação.
- **A3.** Gerador de carga rodando continuamente a partir da filial.

Z. Zabbix (foco: rede e infraestrutura)

- **Z1.** Zabbix Server, frontend e banco em contêineres na LAN Gerência, na versão LTS mais recente.
- **Z2.** Zabbix Agent 2 nos hosts Linux da matriz (dns, web, api, db), usando templates de Linux, Nginx e PostgreSQL.
- **Z3.** Roteadores monitorados por SNMP, com descoberta automática (LLD) das interfaces: tráfego, erros e estado operacional de cada uma.
- **Z4.** Monitoramento ICMP (ping, perda e latência) dos roteadores, do enlace WAN e dos hosts da filial.
- **Z5.** Cenário web com ao menos 2 passos (abrir o catálogo e criar um pedido), verificando código de resposta e conteúdo.
- **Z6.** Triggers com severidades adequadas para: enlace WAN fora, host indisponível, serviço parado, CPU alta e loja fora. Com **dependências**: se o r-filial cair, os alertas dos hosts da filial não devem disparar em cascata.

- **Z7.** Mapa de rede com a topologia real, mostrando o estado dos hosts e o tráfego dos enlaces.
- **Z8.** Dashboard da infraestrutura e notificação por um canal real (Telegram, e-mail ou Discord).

P. Prometheus + Grafana (foco: aplicação e contêineres)

- **P1.** Prometheus coletando: node_exporter nos hosts, cAdvisor (contêineres), `/metrics` da api, postgres_exporter e nginx exporter.
- **P2.** Blackbox exporter na filial com sondas HTTP (loja), DNS e ICMP (roteadores).
- **P3.** Regras de alerta em arquivo, com `for:` e labels de severidade, e Alertmanager enviando notificações agrupadas para um canal real.
- **P4.** Dashboards do Grafana provisionados como código (datasources e dashboards em arquivos): um de infraestrutura (método USE), um da aplicação (método RED) e um de negócio (pedidos por minuto, faturamento).

O. Observabilidade

- **O1.** Loki recebendo os logs dos contêineres (via Grafana Alloy ou Promtail), consultáveis no Grafana.
- **O2.** Correlação: a partir de um pico de erros no painel da aplicação, chegar aos logs daquele intervalo com um clique (link de dados ou painel de logs filtrado).
- **O3.** Dois SLOs para a loja, definidos no relatório e calculados no Grafana. Sugestão: disponibilidade de 99,5% e latência p95 abaixo de 300 ms. O painel deve mostrar o orçamento de erro restante.

Barema de avaliação

O trabalho vale 100 pontos, mais até 10 de bônus. Todo item é verificado **ao vivo** no ambiente da dupla. A coluna "Obtido" fica livre para o professor preencher durante a apresentação.

Bloco	Item	Como verificar na apresentação	Pontos	Obtido
Infraestrutura (15)	I1. Roteamento estático	`ip route` nos dois roteadores confere com a tabela de rotas; `traceroute` da filial até a loja passa pelos dois roteadores	5	

Bloco	Item	Como verificar na apresentação	Pontos	Obtido
Infraestrutura (15)	I2. DNS interno	`dig loja.cafeplanalto.lab` a partir da filial	3	
Infraestrutura (15)	I3. Ambiente reproduzível	Professor pede para destruir e subir o ambiente; tudo volta, rotas inclusive, sem passos manuais	5	
Infraestrutura (15)	I4. Diagrama, endereçamento e rotas	Documentação confere com o ambiente real	2	
Aplicação (10)	A1. Loja em três camadas	Pedido criado pelo navegador ou curl a partir da filial	3	
Aplicação (10)	A2. Instrumentação própria	`/metrics` com as métricas exigidas, logs em JSON, `/health` e `/admin/caos` funcionando	5	
Aplicação (10)	A3. Carga contínua	Tráfego visível nos painéis sem intervenção da dupla	2	
Zabbix (25)	Z1 e Z2. Servidor e agentes	Hosts com disponibilidade verde e templates corretos	5	
Zabbix (25)	Z3. SNMP e LLD de interfaces	Interfaces dos roteadores descobertas automaticamente, com tráfego e estado	5	
Zabbix (25)	Z4. ICMP	Gráficos de latência e perda do enlace WAN	3	
Zabbix (25)	Z5. Cenário web	Dois passos com verificação de código e conteúdo	2	
Zabbix (25)	Z6. Triggers e dependências	Queda do r-filial gera um único problema, sem cascata	4	
Zabbix (25)	Z7. Mapa de rede	Mapa fiel à topologia, mudando de cor durante uma falha	3	

Bloco	Item	Como verificar na apresentação	Pontos	Obtido
Zabbix (25)	Z8. Dashboard e notificação	Mensagem chega ao celular durante a apresentação	3	
Prometheus + Grafana (20)	P1. Alvos de coleta	Página Targets com todos os alvos UP	5	
Prometheus + Grafana (20)	P2. Blackbox na filial	Sondas HTTP, DNS e ICMP com `probe_success`	4	
Prometheus + Grafana (20)	P3. Alertas e Alertmanager	Alerta dispara, notifica e se resolve sozinho após a recuperação	5	
Prometheus + Grafana (20)	P4. Dashboards USE, RED e negócio	Dashboards carregados de arquivos do repositório	6	
Observabilidade (10)	O1. Logs no Loki	Consulta LogQL filtrando erros da api	3	
Observabilidade (10)	O2. Correlação métrica e log	Do pico de erros aos logs do intervalo com um clique	3	
Observabilidade (10)	O3. SLOs e orçamento de erro	Painel com SLI, meta e orçamento restante	4	
Falhas ao vivo (12)	Falha sorteada 1	Detectou nas duas ferramentas (3), localizou a causa (2), mostrou a recuperação (1)	6	
Falhas ao vivo (12)	Falha sorteada 2	Mesmos critérios	6	
Relatório e repositório (8)	Relatório comparativo	Comparação fundamentada no que a dupla observou, com recomendação final	5	
Relatório e repositório (8)	README e organização	Outra pessoa consegue subir o ambiente seguindo o README	3	
Total			**100**	

Bônus (até 10 pontos)

Item	Pontos
Traces com OpenTelemetry e Tempo ou Jaeger, com link do trace para os logs	4
Zabbix Proxy na filial, com os dados recuperados após uma queda da WAN	2
Alerta por taxa de consumo do orçamento de erro (burn rate)	2
Plugin do Zabbix no Grafana: um painel único com dados das duas ferramentas	2
Firewall com nftables no r-matriz: só a LAN Gerência acessa SSH e SNMP	1
Regra de gravação (recording rule) usada em painel ou alerta	1
SNMPv3 com autenticação e criptografia	1

Cálculo da nota

Nota da dupla = mínimo entre 100 e (pontos + bônus), dividido por 10.

Nota individual = nota da dupla × fator de arguição. Cada aluno responde a uma pergunta sobre uma parte que **não** apresentou:

Desempenho na arguição	Fator
Explica com segurança e mostra no ambiente	1,0
Explica parcialmente	0,8
Não demonstra conhecimento do trabalho	0,5

Regras gerais. Item mostrado apenas por print ou vídeo, e não ao vivo, vale metade. Configuração feita na interface e não versionada vale metade. Se o ambiente não subir no dia, a dupla apresenta com evidências gravadas e todos os itens valem metade.

Apresentação, entregáveis e cronograma

Cada dupla tem 20 minutos com o ambiente rodando. O professor conduz a parte de falhas e a arguição; o resto é conduzido pela dupla.

Roteiro (20 minutos)

1. **Arquitetura (3 min):** diagrama, plano de endereçamento e tabela de rotas.
2. **Verificação da infraestrutura (3 min):** rotas, `traceroute`, DNS e loja funcionando a partir da filial.
3. **Zabbix (4 min):** hosts, LLD, mapa, triggers, dashboard.
4. **Prometheus + Grafana (4 min):** alvos, dashboards USE, RED e negócio, Loki, SLOs.
5. **Falhas ao vivo (4 min):** o professor sorteia 2 falhas do catálogo abaixo e as injeta. A dupla narra o que cada ferramenta mostra, localiza a causa e acompanha a recuperação.
6. **Arguição individual (2 min):** uma pergunta para cada aluno.

Catálogo de falhas

As duplas conhecem o catálogo com antecedência e devem ensaiar todos os cenários. Os comandos supõem nomes de nós do Containerlab no formato `clab-cafeplanalto-<nó>`.

#	Falha	Como o professor injeta	O que deve aparecer
1	Queda do enlace WAN	`docker exec clab-cafeplanalto-r-filial ip link set eth1 down`	Interface down via SNMP; hosts da filial inacessíveis, com dependências evitando cascata; Prometheus perde a sonda (`up == 0`)
2	Rota estática removida	`docker exec clab-cafeplanalto-r-matriz ip route del 172.20.20.0/24`	Enlace continua up no SNMP, mas o ICMP para a filial falha e a sonda some. A dupla deve explicar por que isso não é queda de enlace
3	Latência e perda na WAN	`tc qdisc add dev eth1 root netem delay 300ms loss 5%` no r-filial	Latência e perda no ICMP do Zabbix; duração da sonda no blackbox; p95 da loja visto pela filial sobe

#	Falha	Como o professor injeta	O que deve aparecer
4	Banco de dados parado	`docker stop clab-cafeplanalto-db`	Trigger do PostgreSQL; `/health` falha; taxa de 5xx sobe; logs de erro de conexão no Loki
5	API lenta	`POST /admin/caos` com `latencia_ms=800`	p95 acima do SLO; alerta de latência; cenário web lento no Zabbix
6	Erros na API	`POST /admin/caos` com `taxa_erro=0.2`	Taxa de erros no painel RED; orçamento de erro caindo; logs com status 500
7	DNS fora	`docker stop clab-cafeplanalto-dns`	Sonda DNS falha; item `net.dns` do Zabbix; loja inacessível por nome

O professor pode combinar duas falhas ao mesmo tempo, como 2 e 4, para testar se a dupla separa causa e sintoma.

Entregáveis

Entrega no **Google Classroom**, na tarefa da disciplina, até o fim da semana 3. Todas as duplas entregam no mesmo prazo, qualquer que seja o dia em que apresentam:

- **Link do repositório Git** com: topologia do Containerlab, Dockerfiles, regras do Prometheus e do Alertmanager, dashboards do Grafana em JSON, templates do Zabbix exportados, código da aplicação e README com o passo a passo para subir o ambiente.
- **Relatório em PDF (6 a 10 páginas)** com: diagrama, plano de endereçamento e tabela de rotas; decisões de projeto; SLOs definidos e justificados; resultado do ensaio de cada falha do catálogo; comparação entre Zabbix e Prometheus + Grafana (modelo de coleta, curva de aprendizado, monitoramento de rede, monitoramento de aplicação, alertas, consumo de recursos); e recomendação final à diretoria da Café Planalto.

Cronograma sugerido (4 semanas)

1. **Semana 1:** formar as duplas, desenhar a topologia, o endereçamento e a tabela de rotas, preparar a VM e subir roteadores, LANs, DNS e a primeira versão da loja. *Checkpoint 1:* `traceroute` da filial até a LAN Serviços passando pelos dois roteadores.

2. **Semana 2:** instrumentar a loja, ligar o gerador de carga e montar o Zabbix completo (agentes, SNMP, ICMP, cenário web, triggers, mapa, notificação). *Checkpoint 2:* mapa de rede e um alerta chegando ao celular.
3. **Semana 3:** Prometheus, exporters, Alertmanager, Grafana, Loki e SLOs; ensaio das falhas do catálogo; relatório. Entrega no Classroom até o fim da semana.
4. **Semana 4:** apresentações, em ordem sorteada. Se a turma não couber nas aulas da semana 4, as apresentações continuam na semana seguinte, sem mudar o prazo de entrega.

Os checkpoints não valem nota. Eles servem para o professor identificar duplas travadas cedo.

Dicas e referências

Armadilhas comuns

- **Coletar pela rede de gerência.** Se Zabbix ou Prometheus alcançam os hosts pelo eth0 do Containerlab, a queda da WAN não aparece. Confira as rotas de cada nó.
- **Dashboards feitos só clicando.** Eles somem quando o contêiner do Grafana é recriado. Use provisionamento desde o início.
- **Relógios dessincronizados.** Métricas e logs de hosts com horários diferentes não se correlacionam. Todos os contêineres usam o relógio da VM; mantenha a VM sincronizada por NTP.
- **Labels de alta cardinalidade.** ID de pedido ou de usuário como label derruba o Prometheus. Isso vai para os logs.
- **Alertas que ninguém consegue agir.** "CPU acima de 80%" sozinho gera ruído. Prefira alertas de sintoma (a loja está lenta ou com erro) e use os de causa para o diagnóstico.
- **Memória da VM.** O banco do Zabbix e o Prometheus são os maiores consumidores. Limite a retenção do Prometheus (por exemplo, `--storage.tsdb.retention.time=7d`).
- **Imagens sem arm64.** Em Mac com Apple Silicon, confira a arquitetura da imagem antes de adotá-la.

Referências

- Documentação do Zabbix: agentes, SNMP, LLD, dependências de triggers e mapas.
- Documentação do Prometheus: coleta, PromQL, regras de alerta, Alertmanager.
- Documentação do Grafana: provisionamento, Loki, Tempo e correlações.
- Containerlab: instalação, topologias e nós do tipo linux e bridge.

- Manual do `ip route`: rotas estáticas no Linux.
- Google SRE Book, capítulo "Monitoring Distributed Systems": os quatro sinais dourados.
- Google SRE Workbook, capítulo "Alerting on SLOs": SLOs e alertas por taxa de consumo (bônus).
- Método USE, de Brendan Gregg: utilização, saturação e erros de recursos.
- OpenTelemetry: instrumentação de traces (bônus).